Name	K Dhyana samaga
Registration Number	261091010031
Submission Date	18/08/2026

1.Caesar Cipher Attack

```
def brutforce():
  cipher_text = input("Enter the cipher text: ")
  print(f"Ciphertext: {cipher_text}\n")
  print("--- Brute Force Results ---")

  cipher_text = cipher_text.lower()

  for key in range(26):
    plain_text = ""
    for char in cipher_text:
      if char.isalpha():
        ord_num = ord(char) - ord('a')
       decrypted_char = (ord_num - key) % 26
       plain_text = plain_text + chr(decrypted_char + ord('a'))
     else:
        plain_text = plain_text + char

    print(f"Key {key:2d} -> {plain_text}")
```

Output

```
┌──(student㉿PC-36)-[~/Documents/K Dhyana/ACL/LAB-2]
└─$ python main.py
=== Classical Cryptanalysis Tool ===
1. Caesar Brute Force
2. Multiplicative Brute Force
3. Affine Brute Force
4. Hill Brute Force
5. Monoalphabetic Frequency Analysis

Select an option (1-4): 1
Enter the cipher text: mjqqt
Ciphertext: mjqqt

--- Brute Force Results ---
Key  0 -> mjqqt
Key  1 -> lipps
Key  2 -> khoor
Key  3 -> jgnnq
Key  4 -> ifmmp
Key  5 -> hello
Key  6 -> gdkkn
Key  7 -> fcjjm
Key  8 -> ebiil
Key  9 -> dahhk
Key 10 -> czggj
Key 11 -> byffi
Key 12 -> axeeh
Key 13 -> zwddg
Key 14 -> yvccf
Key 15 -> xubbe
Key 16 -> wtaad
Key 17 -> vszzc
Key 18 -> uryyb
Key 19 -> tqxxa
Key 20 -> spwwz
Key 21 -> rovvy
Key 22 -> qnuux
Key 23 -> pmttw
Key 24 -> olssv
Key 25 -> nkrru

┌──(student㉿PC-36)-[~/Documents/K Dhyana/ACL/LAB-2]
└─$
```

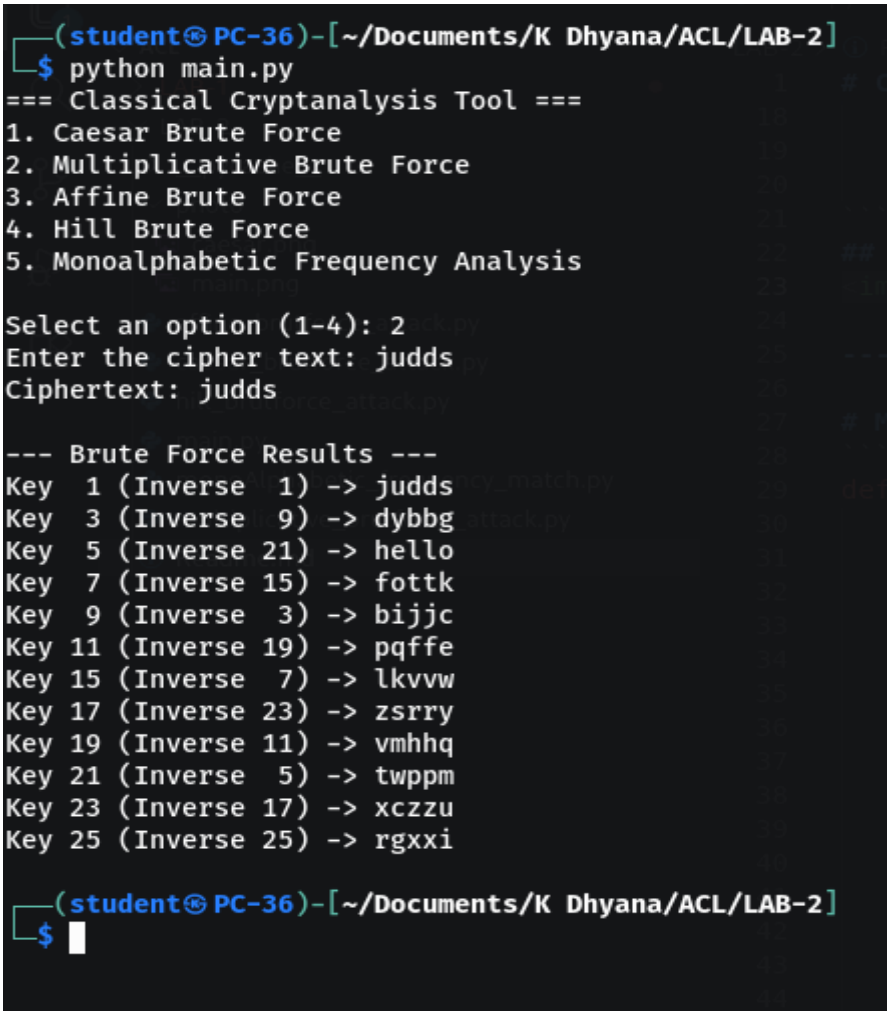
2.Multiplicative Cipher Attack

```
def brutforce():
cipher_text = input("Enter the cipher text: ")
inverse_map = {1: 1,3: 9,5: 21,7: 15,9: 3,11: 19,15: 7,17: 23,19: 11,21: 5,23: 17,25: 25}
print(f"Ciphertext: {cipher_text}\n")
print("--- Brute Force Results ---")

cipher_text = cipher_text.lower()
for key,inv_key in inverse_map.items():
plain_text = ""
for char in cipher_text:
if char.isalpha():
ord_num = ord(char) - ord('a')
pln_num = (ord_num * inv_key) % 26
plain_text = plain_text + chr(pln_num + ord('a'))
else:
plain_text = plain_text + char

print(f"Key {key:2d} (Inverse {inv_key:2d}) -> {plain_text}")
```

Output



3.Affine Cipher Attack

```
def brutforce():
cipher_text = input("Enter the cipher text: ")
inverse_map = {1: 1,3: 9,5: 21,7: 15,9: 3,11: 19,15: 7,17: 23,19: 11,21: 5,23: 17,25: 25}
print(f"Ciphertext: {cipher_text}\n")
print("--- Brute Force Results ---")

cipher_text = cipher_text.lower()

for key1,inv_key in inverse_map.items():
for key2 in range(26):
plain_text = ""

for char in cipher_text:
if char.isalpha():
ord_num = ord(char) - ord('a')
temp = (inv_key * (ord_num - key2)) % 26
plain_text = plain_text + chr(temp + ord('a'))
else:
plain_text = plain_text + char
print(f"Key a={key1:2d} (Inv={inv_key:2d}), b={key2:2d} -> {plain_text}")
```

Output

```
┌──(student㉿PC-36)-[~/Documents/K Dhyana/ACL/LAB-2]
└─$ python main.py
=== Classical Cryptanalysis Tool ===
1. Caesar Brute Force
2. Multiplicative Brute Force
3. Affine Brute Force
4. Hill Brute Force
5. Monoalphabetic Frequency Analysis

Select an option (1-4): 3
Enter the cipher text: tennc
Ciphertext: tennc

--- Brute Force Results ---
Key a= 1 (Inv= 1), b= 0 -> tennc
Key a= 1 (Inv= 1), b= 1 -> sdmmb
Key a= 1 (Inv= 1), b= 2 -> rclla
Key a= 1 (Inv= 1), b= 3 -> qbkkz
Key a= 1 (Inv= 1), b= 4 -> pajjy
Key a= 1 (Inv= 1), b= 5 -> oziix
Key a= 1 (Inv= 1), b= 6 -> nyhhw
Key a= 1 (Inv= 1), b= 7 -> mxggv
Key a= 1 (Inv= 1), b= 8 -> lwffu
Key a= 1 (Inv= 1), b= 9 -> kveet
Key a= 1 (Inv= 1), b=10 -> judds
Key a= 1 (Inv= 1), b=11 -> itccr
Key a= 1 (Inv= 1), b=12 -> hsbbq
Key a= 1 (Inv= 1), b=13 -> graap
Key a= 1 (Inv= 1), b=14 -> fqzzo
Key a= 1 (Inv= 1), b=15 -> epyyn
Key a= 1 (Inv= 1), b=16 -> doxxm
```

```
Key a= 1 (Inv= 1), b=16 -> doxxm
Key a= 1 (Inv= 1), b=17 -> cnwwl
Key a= 1 (Inv= 1), b=18 -> bmvvk
Key a= 1 (Inv= 1), b=19 -> aluuj
Key a= 1 (Inv= 1), b=20 -> zktti
Key a= 1 (Inv= 1), b=21 -> yjssh
Key a= 1 (Inv= 1), b=22 -> xirrg
Key a= 1 (Inv= 1), b=23 -> whqqf
Key a= 1 (Inv= 1), b=24 -> vgppe
Key a= 1 (Inv= 1), b=25 -> ufood
Key a= 3 (Inv= 9), b= 0 -> pknns
Key a= 3 (Inv= 9), b= 1 -> gbeej
Key a= 3 (Inv= 9), b= 2 -> xsvva
Key a= 3 (Inv= 9), b= 3 -> ojmmr
Key a= 3 (Inv= 9), b= 4 -> faddi
Key a= 3 (Inv= 9), b= 5 -> wruuz
Key a= 3 (Inv= 9), b= 6 -> nillq
Key a= 3 (Inv= 9), b= 7 -> ezcch
Key a= 3 (Inv= 9), b= 8 -> vqtty
Key a= 3 (Inv= 9), b= 9 -> mhkkp
Key a= 3 (Inv= 9), b=10 -> dybbg
Key a= 3 (Inv= 9), b=11 -> upssx
Key a= 3 (Inv= 9), b=12 -> lgjjo
Key a= 3 (Inv= 9), b=13 -> cxaaf
```

```
Key a= 3 (Inv= 9), b=14 -> torrw
Key a= 3 (Inv= 9), b=15 -> kfiin
Key a= 3 (Inv= 9), b=16 -> bwzze
Key a= 3 (Inv= 9), b=17 -> snqqv
Key a= 3 (Inv= 9), b=18 -> jehhm
Key a= 3 (Inv= 9), b=19 -> avyyd
Key a= 3 (Inv= 9), b=20 -> rmppu
Key a= 3 (Inv= 9), b=21 -> idggl
Key a= 3 (Inv= 9), b=22 -> zuxxc
Key a= 3 (Inv= 9), b=23 -> qloot
Key a= 3 (Inv= 9), b=24 -> hcffk
Key a= 3 (Inv= 9), b=25 -> ytwwb
Key a= 5 (Inv=21), b= 0 -> jgnnq
Key a= 5 (Inv=21), b= 1 -> olssv
Key a= 5 (Inv=21), b= 2 -> tqxxa
Key a= 5 (Inv=21), b= 3 -> yvccf
Key a= 5 (Inv=21), b= 4 -> dahhk
Key a= 5 (Inv=21), b= 5 -> ifmmp
Key a= 5 (Inv=21), b= 6 -> nkrru
Key a= 5 (Inv=21), b= 7 -> spwwz
Key a= 5 (Inv=21), b= 8 -> xubbe
Key a= 5 (Inv=21), b= 9 -> czggj
Key a= 5 (Inv=21), b=10 -> hello
Key a= 5 (Inv=21), b=11 -> mjqqt
Key a= 5 (Inv=21), b=12 -> rovvy
Key a= 5 (Inv=21), b=13 -> wtaad
Key a= 5 (Inv=21), b=14 -> byffi
Key a= 5 (Inv=21), b=15 -> gdkkn
Key a= 5 (Inv=21), b=16 -> lipps
Key a= 5 (Inv=21), b=17 -> qnuux
Key a= 5 (Inv=21), b=18 -> vszzc
Key a= 5 (Inv=21), b=19 -> axeeh
Key a= 5 (Inv=21), b=20 -> fcjjm
Key a= 5 (Inv=21), b=21 -> khoor
Key a= 5 (Inv=21), b=22 -> pmttw
Key a= 5 (Inv=21), b=23 -> uryyb
```

```
Key a= 5 (Inv=21), b=21 -> khoor
Key a= 5 (Inv=21), b=22 -> pmttw
Key a= 5 (Inv=21), b=23 -> uryyb
Key a= 5 (Inv=21), b=24 -> zwddg
Key a= 5 (Inv=21), b=25 -> ebiil
Key a= 7 (Inv=15), b= 0 -> zinne
Key a= 7 (Inv=15), b= 1 -> ktyyp
Key a= 7 (Inv=15), b= 2 -> vejja
Key a= 7 (Inv=15), b= 3 -> gpuul
Key a= 7 (Inv=15), b= 4 -> raffw
Key a= 7 (Inv=15), b= 5 -> clqqh
Key a= 7 (Inv=15), b= 6 -> nwbbs
Key a= 7 (Inv=15), b= 7 -> yhmmd
Key a= 7 (Inv=15), b= 8 -> jsxxo
Key a= 7 (Inv=15), b= 9 -> udiiz
Key a= 7 (Inv=15), b=10 -> fottk
Key a= 7 (Inv=15), b=11 -> qzeev
Key a= 7 (Inv=15), b=12 -> bkppg
Key a= 7 (Inv=15), b=13 -> mvaar
Key a= 7 (Inv=15), b=14 -> xgllc
Key a= 7 (Inv=15), b=15 -> irwwn
Key a= 7 (Inv=15), b=16 -> tchhy
Key a= 7 (Inv=15), b=17 -> enssj
Key a= 7 (Inv=15), b=18 -> pyddu
Key a= 7 (Inv=15), b=19 -> ajoof
Key a= 7 (Inv=15), b=20 -> luzzq
Key a= 7 (Inv=15), b=21 -> wfkkb
Key a= 7 (Inv=15), b=22 -> hqvvm
Key a= 7 (Inv=15), b=23 -> sbggx
Key a= 7 (Inv=15), b=24 -> dmrri
Key a= 7 (Inv=15), b=25 -> oxcct
Key a= 9 (Inv= 3), b= 0 -> fmnng
Key a= 9 (Inv= 3), b= 1 -> cjkkd
Key a= 9 (Inv= 3), b= 2 -> zghha
Key a= 9 (Inv= 3), b= 3 -> wdeex
```

```
Key a= 3 (Inv= 9), b=23 -> qloot
Key a= 3 (Inv= 9), b=24 -> hcffk
Key a= 3 (Inv= 9), b=25 -> ytwwb
Key a= 5 (Inv=21), b= 0 -> jgnnq
Key a= 5 (Inv=21), b= 1 -> olssv
Key a= 5 (Inv=21), b= 2 -> tqxxa
Key a= 5 (Inv=21), b= 3 -> yvccf
Key a= 5 (Inv=21), b= 4 -> dahhk
Key a= 5 (Inv=21), b= 5 -> ifmmp
Key a= 5 (Inv=21), b= 6 -> nkrru
Key a= 5 (Inv=21), b= 7 -> spwwz
Key a= 5 (Inv=21), b= 8 -> xubbe
Key a= 5 (Inv=21), b= 9 -> czggj
Key a= 5 (Inv=21), b=10 -> hello
Key a= 5 (Inv=21), b=11 -> mjqqt
Key a= 5 (Inv=21), b=12 -> rovvy
Key a= 5 (Inv=21), b=13 -> wtaad
Key a= 5 (Inv=21), b=14 -> byffi
Key a= 5 (Inv=21), b=15 -> gdkkn
Key a= 5 (Inv=21), b=16 -> lipps
Key a= 5 (Inv=21), b=17 -> qnuux
Key a= 5 (Inv=21), b=18 -> vszzc
Key a= 5 (Inv=21), b=19 -> axeeh
Key a= 5 (Inv=21), b=20 -> fcjjm
Key a= 5 (Inv=21), b=21 -> khoor
Key a= 5 (Inv=21), b=22 -> pmttw
Key a= 5 (Inv=21), b=23 -> uryyb
Key a= 5 (Inv=21), b=24 -> zwddg
Key a= 5 (Inv=21), b=25 -> ebiil
Key a= 7 (Inv=15), b= 0 -> zinne
Key a= 7 (Inv=15), b= 1 -> ktyyp
Key a= 7 (Inv=15), b= 2 -> vejja
Key a= 7 (Inv=15), b= 3 -> gpuul
Key a= 7 (Inv=15), b= 4 -> raffw
Key a= 7 (Inv=15), b= 5 -> clqqh
```

```
Key a= 9 (Inv= 3), b=23 -> ovwwp
Key a= 9 (Inv= 3), b=24 -> lsttm
Key a= 9 (Inv= 3), b=25 -> ipqqj
Key a=11 (Inv=19), b= 0 -> xynnm
Key a=11 (Inv=19), b= 1 -> efuut
Key a=11 (Inv=19), b= 2 -> lmbba
Key a=11 (Inv=19), b= 3 -> stiih
Key a=11 (Inv=19), b= 4 -> zappo
Key a=11 (Inv=19), b= 5 -> ghwwv
Key a=11 (Inv=19), b= 6 -> noddc
Key a=11 (Inv=19), b= 7 -> uvkkj
Key a=11 (Inv=19), b= 8 -> bcrrq
Key a=11 (Inv=19), b= 9 -> ijyyx
Key a=11 (Inv=19), b=10 -> pqffe
Key a=11 (Inv=19), b=11 -> wxmml
Key a=11 (Inv=19), b=12 -> detts
Key a=11 (Inv=19), b=13 -> klaaz
Key a=11 (Inv=19), b=14 -> rshhg
Key a=11 (Inv=19), b=15 -> yzoon
Key a=11 (Inv=19), b=16 -> fgvvu
Key a=11 (Inv=19), b=17 -> mnccb
Key a=11 (Inv=19), b=18 -> tujji
Key a=11 (Inv=19), b=19 -> abqqp
Key a=11 (Inv=19), b=20 -> hixxw
Key a=11 (Inv=19), b=21 -> opeed
Key a=11 (Inv=19), b=22 -> vwllk
Key a=11 (Inv=19), b=23 -> cdssr
Key a=11 (Inv=19), b=24 -> jkzzy
Key a=11 (Inv=19), b=25 -> qrggf
Key a=15 (Inv= 7), b= 0 -> dcnno
Key a=15 (Inv= 7), b= 1 -> wvggh
Key a=15 (Inv= 7), b= 2 -> pozza
Key a=15 (Inv= 7), b= 3 -> ihsst
Key a=15 (Inv= 7), b= 4 -> ballm
Key a=15 (Inv= 7), b= 5 -> uteef
```

```
Key a=15 (Inv= 7), b=13 -> qpaab
Key a=15 (Inv= 7), b=14 -> jittu
Key a=15 (Inv= 7), b=15 -> cbmmn
Key a=15 (Inv= 7), b=16 -> vuffg
Key a=15 (Inv= 7), b=17 -> onyyz
Key a=15 (Inv= 7), b=18 -> hgrrs
Key a=15 (Inv= 7), b=19 -> azkkl
Key a=15 (Inv= 7), b=20 -> tsdde
Key a=15 (Inv= 7), b=21 -> mlwwx
Key a=15 (Inv= 7), b=22 -> feppq
Key a=15 (Inv= 7), b=23 -> yxiij
Key a=15 (Inv= 7), b=24 -> rqbbc
Key a=15 (Inv= 7), b=25 -> kjuuv
Key a=17 (Inv=23), b= 0 -> vonnu
Key a=17 (Inv=23), b= 1 -> yrqqx
Key a=17 (Inv=23), b= 2 -> butta
Key a=17 (Inv=23), b= 3 -> exwwd
Key a=17 (Inv=23), b= 4 -> hazzg
Key a=17 (Inv=23), b= 5 -> kdccj
Key a=17 (Inv=23), b= 6 -> ngffm
Key a=17 (Inv=23), b= 7 -> qjiip
Key a=17 (Inv=23), b= 8 -> tmlls
Key a=17 (Inv=23), b= 9 -> wpoov
Key a=17 (Inv=23), b=10 -> zsrry
Key a=17 (Inv=23), b=11 -> cvuub
Key a=17 (Inv=23), b=12 -> fyxxe
Key a=17 (Inv=23), b=13 -> ibaah
Key a=17 (Inv=23), b=14 -> leddk
Key a=17 (Inv=23), b=15 -> ohggn
Key a=17 (Inv=23), b=16 -> rkjjq
Key a=17 (Inv=23), b=17 -> unmmt
Key a=17 (Inv=23), b=18 -> xqppw
Key a=17 (Inv=23), b=19 -> atssz
Key a=17 (Inv=23), b=20 -> dwvvc
Key a=17 (Inv=23), b=21 -> gzyyf
```

```
Key a=21 (Inv= 5), b= 7 -> ileeb
Key a=21 (Inv= 5), b= 8 -> dgzzw
Key a=21 (Inv= 5), b= 9 -> ybuur
Key a=21 (Inv= 5), b=10 -> twppm
Key a=21 (Inv= 5), b=11 -> orkkh
Key a=21 (Inv= 5), b=12 -> jmffc
Key a=21 (Inv= 5), b=13 -> ehaax
Key a=21 (Inv= 5), b=14 -> zcvvs
Key a=21 (Inv= 5), b=15 -> uxqqn
Key a=21 (Inv= 5), b=16 -> pslli
Key a=21 (Inv= 5), b=17 -> knggd
Key a=21 (Inv= 5), b=18 -> fibby
Key a=21 (Inv= 5), b=19 -> adwwt
Key a=21 (Inv= 5), b=20 -> vyrro
Key a=21 (Inv= 5), b=21 -> qtmmj
Key a=21 (Inv= 5), b=22 -> lohhe
Key a=21 (Inv= 5), b=23 -> gjccz
Key a=21 (Inv= 5), b=24 -> bexxu
Key a=21 (Inv= 5), b=25 -> wzssp
Key a=23 (Inv=17), b= 0 -> lqnni
Key a=23 (Inv=17), b= 1 -> uzwwr
Key a=23 (Inv=17), b= 2 -> diffa
Key a=23 (Inv=17), b= 3 -> mrooj
Key a=23 (Inv=17), b= 4 -> vaxxs
Key a=23 (Inv=17), b= 5 -> ejggb
Key a=23 (Inv=17), b= 6 -> nsppk
Key a=23 (Inv=17), b= 7 -> wbyyt
Key a=23 (Inv=17), b= 8 -> fkhhc
Key a=23 (Inv=17), b= 9 -> otqql
Key a=23 (Inv=17), b=10 -> xczzu
Key a=23 (Inv=17), b=11 -> gliid
Key a=23 (Inv=17), b=12 -> purrm
Key a=23 (Inv=17), b=13 -> ydaav
Key a=23 (Inv=17), b=14 -> hmjje
Key a=23 (Inv=17), b=15 -> qvssn
```


```
Key a=21 (Inv= 5), b= 7 -> ileeb
Key a=21 (Inv= 5), b= 8 -> dgzzw
Key a=21 (Inv= 5), b= 9 -> ybuur
Key a=21 (Inv= 5), b=10 -> twppm
Key a=21 (Inv= 5), b=11 -> orkkh
Key a=21 (Inv= 5), b=12 -> jmffc
Key a=21 (Inv= 5), b=13 -> ehaax
Key a=21 (Inv= 5), b=14 -> zcvvs
Key a=21 (Inv= 5), b=15 -> uxqqn
Key a=21 (Inv= 5), b=16 -> pslli
Key a=21 (Inv= 5), b=17 -> knggd
Key a=21 (Inv= 5), b=18 -> fibby
Key a=21 (Inv= 5), b=19 -> adwwt
Key a=21 (Inv= 5), b=20 -> vyrro
Key a=21 (Inv= 5), b=21 -> qtmmj
Key a=21 (Inv= 5), b=22 -> lohhe
Key a=21 (Inv= 5), b=23 -> gjccz
Key a=21 (Inv= 5), b=24 -> bexxu
Key a=21 (Inv= 5), b=25 -> wzssp
Key a=23 (Inv=17), b= 0 -> lqnni
Key a=23 (Inv=17), b= 1 -> uzwwr
Key a=23 (Inv=17), b= 2 -> diffa
Key a=23 (Inv=17), b= 3 -> mrooj
Key a=23 (Inv=17), b= 4 -> vaxxs
Key a=23 (Inv=17), b= 5 -> ejggb
Key a=23 (Inv=17), b= 6 -> nsppk
Key a=23 (Inv=17), b= 7 -> wbyyt
Key a=23 (Inv=17), b= 8 -> fkhhc
Key a=23 (Inv=17), b= 9 -> otqql
Key a=23 (Inv=17), b=10 -> xczzu
Key a=23 (Inv=17), b=11 -> gliid
Key a=23 (Inv=17), b=12 -> purrm
Key a=23 (Inv=17), b=13 -> ydaav
Key a=23 (Inv=17), b=14 -> hmjje
Key a=23 (Inv=17), b=15 -> qvssn
```


```
Key a=23 (Inv=17), b=20 -> jottg
Key a=23 (Inv=17), b=21 -> sxuup
Key a=23 (Inv=17), b=22 -> bgddy
Key a=23 (Inv=17), b=23 -> kpmmh
Key a=23 (Inv=17), b=24 -> tyvvq
Key a=23 (Inv=17), b=25 -> cheez
Key a=25 (Inv=25), b= 0 -> hwnny
Key a=25 (Inv=25), b= 1 -> ixooz
Key a=25 (Inv=25), b= 2 -> jyppa
Key a=25 (Inv=25), b= 3 -> kzqqb
Key a=25 (Inv=25), b= 4 -> larrc
Key a=25 (Inv=25), b= 5 -> mbssd
Key a=25 (Inv=25), b= 6 -> nctte
Key a=25 (Inv=25), b= 7 -> oduuf
Key a=25 (Inv=25), b= 8 -> pevvg
Key a=25 (Inv=25), b= 9 -> qfwwh
Key a=25 (Inv=25), b=10 -> rgxxi
Key a=25 (Inv=25), b=11 -> shyyj
Key a=25 (Inv=25), b=12 -> tizzk
Key a=25 (Inv=25), b=13 -> ujaal
Key a=25 (Inv=25), b=14 -> vkbbm
Key a=25 (Inv=25), b=15 -> wlccn
Key a=25 (Inv=25), b=16 -> xmddo
Key a=25 (Inv=25), b=17 -> yneep
Key a=25 (Inv=25), b=18 -> zoffq
Key a=25 (Inv=25), b=19 -> apggr
Key a=25 (Inv=25), b=20 -> bqhhs
Key a=25 (Inv=25), b=21 -> criit
Key a=25 (Inv=25), b=22 -> dsjju
Key a=25 (Inv=25), b=23 -> etkkv
Key a=25 (Inv=25), b=24 -> fullw
Key a=25 (Inv=25), b=25 -> gvmmx

┌──(student㉿PC-36)-[~/Documents/K Dhyana/ACL/LAB-2]
└─$
```

4.Hill Cipher Attack

```
import numpy as np

def mod_inverse(det, modulus=26):
det = det % modulus
for x in range(1, modulus):
if (det * x) % modulus == 1:
return x
return None

def brutforce():
ciphertext = input("Enter ciphertext: ").strip()
ciphertext = ciphertext.lower().replace(" ", "")
if len(ciphertext) % 2 != 0:
ciphertext += 'x'

print("--- Searching all 456,976 combinations (15,724 valid keys) ---")
common_words = ["the", "and", "ing", "ion", "you", "ent", "that", "for", "this"]
match_count = 0

for a in range(26):
for b in range(26):
for c in range(26):
for d in range(26):
det = (a * d - b * c) % 26
det_inv = mod_inverse(det, 26)
if det_inv is None:
continue

d_mod = (d % 26 + 26) % 26
b_mod = (-b % 26 + 26) % 26
c_mod = (-c % 26 + 26) % 26
a_mod = (a % 26 + 26) % 26
K_inv = (det_inv * np.array([[d_mod, b_mod], [c_mod, a_mod]])) % 26
plaintext = ""
for i in range(0, len(ciphertext), 2):
c1 = ord(ciphertext[i]) - ord('a')
c2 = ord(ciphertext[i+1]) - ord('a')
p1 = (K_inv[0,0] * c1 + K_inv[0,1] * c2) % 26
p2 = (K_inv[1,0] * c1 + K_inv[1,1] * c2) % 26
plaintext += chr(int(p1) + ord('a'))
plaintext += chr(int(p2) + ord('a'))

if any(word in plaintext for word in common_words):
match_count += 1
print(f"Key [[{a},{b}],[{c},{d}]] -> Plaintext: {plaintext}")

print(f"\nBrute-force complete. Found {match_count} potential matches.")
```

Output

```
┌──(K)(student㉿PC-36)-[~/Documents/K Dhyana/ACL/LAB-2]
└─$ python main.py
=== Classical Cryptanalysis Tool ===
1. Caesar Brute Force
2. Multiplicative Brute Force
3. Affine Brute Force
4. Hill Brute Force
5. Monoalphabetic Frequency Analysis

Select an option (1-4): 4
Enter ciphertext: hiozhn
--- Searching all 456,976 combinations (15,724 valid keys) ---
Key [[0,1],[9,6]] -> Plaintext: chforh
Key [[1,0],[5,5]] -> Plaintext: hforhg
Key [[1,0],[9,7]] -> Plaintext: hhothe
Key [[1,0],[12,23]] -> Plaintext: hionhp
Key [[1,2],[12,21]] -> Plaintext: riondp
Key [[1,2],[17,7]] -> Plaintext: thefdc
Key [[1,3],[16,1]] -> Plaintext: hatheb
Key [[1,4],[12,19]] -> Plaintext: bionzp
Key [[1,6],[12,17]] -> Plaintext: lionvp
Key [[1,8],[12,15]] -> Plaintext: vionrp
Key [[1,9],[16,17]] -> Plaintext: handst
Key [[1,10],[12,13]] -> Plaintext: fionnp
Key [[1,11],[0,15]] -> Plaintext: pentun
Key [[1,12],[12,11]] -> Plaintext: pionjp
Key [[1,13],[9,20]] -> Plaintext: uhbthe
Key [[1,14],[12,9]] -> Plaintext: zionfp
Key [[1,15],[10,23]] -> Plaintext: forfgh
Key [[1,16],[12,7]] -> Plaintext: jionbp
Key [[1,18],[12,5]] -> Plaintext: tionxp
Key [[1,20],[12,3]] -> Plaintext: diontp
Key [[1,21],[6,5]] -> Plaintext: rcnfor
Key [[1,22],[12,1]] -> Plaintext: nionpp
Key [[1,24],[12,25]] -> Plaintext: xionlp
Key [[2,1],[11,8]] -> Plaintext: uthatv
Key [[2,11],[11,0]] -> Plaintext: wzhand
Key [[2,13],[7,7]] -> Plaintext: xthekd
Key [[2,13],[7,20]] -> Plaintext: kthexd
Key [[2,17],[21,24]] -> Plaintext: crford
Key [[2,25],[23,0]] -> Plaintext: gfjent
Key [[3,1],[18,15]] -> Plaintext: tcdfor
Key [[3,3],[1,8]] -> Plaintext: ehdthe
Key [[3,4],[11,1]] -> Plaintext: thehdg
Key [[3,5],[8,19]] -> Plaintext: hsthez
Key [[3,5],[20,25]] -> Plaintext: fordgd
Key [[3,6],[15,9]] -> Plaintext: bforzg
Key [[3,9],[22,17]] -> Plaintext: landgt
Key [[3,11],[0,15]] -> Plaintext: fentyn
Key [[3,16],[1,21]] -> Plaintext: rhqthe
Key [[4,3],[1,11]] -> Plaintext: hthegd
Key [[4,3],[1,24]] -> Plaintext: uthetd
Key [[4,7],[7,16]] -> Plaintext: clforv
Key [[4,11],[9,0]] -> Plaintext: yzxand
```

```
Key [[5,11],[15,0]] -> Plaintext: entvnq
Key [[5,12],[25,13]] -> Plaintext: fforng
Key [[5,15],[15,0]] -> Plaintext: entfnk
Key [[5,17],[15,0]] -> Plaintext: entjns
Key [[5,19],[15,0]] -> Plaintext: enttnm
Key [[5,19],[19,22]] -> Plaintext: ghtthe
Key [[5,21],[4,1]] -> Plaintext: forjgp
Key [[5,21],[15,0]] -> Plaintext: entlnw
Key [[5,23],[15,0]] -> Plaintext: entbnc
Key [[5,25],[15,0]] -> Plaintext: entdng
Key [[6,11],[7,0]] -> Plaintext: qzland
Key [[6,19],[21,2]] -> Plaintext: gthejd
Key [[6,19],[21,15]] -> Plaintext: tthewd
Key [[6,23],[19,8]] -> Plaintext: ctforx
Key [[6,25],[17,0]] -> Plaintext: cfdent
Key [[7,8],[25,15]] -> Plaintext: thevdi
Key [[7,9],[8,17]] -> Plaintext: bandkt
Key [[7,9],[11,10]] -> Plaintext: shlthe
Key [[7,9],[18,3]] -> Plaintext: hethep
Key [[7,11],[0,15]] -> Plaintext: renton
Key [[7,11],[14,3]] -> Plaintext: forhgl
Key [[7,13],[16,9]] -> Plaintext: bcpfor
Key [[7,18],[9,17]] -> Plaintext: dfortg
Key [[7,22],[11,23]] -> Plaintext: fhythe
Key [[8,9],[15,6]] -> Plaintext: mtherd
Key [[8,9],[15,19]] -> Plaintext: ztheed
Key [[8,11],[5,0]] -> Plaintext: mzfand
Key [[8,25],[1,0]] -> Plaintext: ifzent
Key [[9,1],[24,5]] -> Plaintext: forrgf
Key [[9,9],[14,17]] -> Plaintext: vandct
Key [[9,10],[19,9]] -> Plaintext: theddy
Key [[9,11],[0,15]] -> Plaintext: tentin
Key [[9,11],[10,21]] -> Plaintext: hcthev
Key [[9,12],[3,11]] -> Plaintext: dhithe
Key [[9,19],[2,19]] -> Plaintext: lcrfor
Key [[9,24],[19,21]] -> Plaintext: zforfg
Key [[9,25],[3,24]] -> Plaintext: qhvthe
Key [[10,3],[17,18]] -> Plaintext: cnforp
Key [[10,11],[3,0]] -> Plaintext: uzrand
Key [[10,25],[9,10]] -> Plaintext: athebd
Key [[10,25],[9,23]] -> Plaintext: ntheod
Key [[10,25],[11,0]] -> Plaintext: wfhent
Key [[11,2],[21,25]] -> Plaintext: xhmthe
Key [[11,4],[3,25]] -> Plaintext: nforpg
Key [[11,9],[20,17]] -> Plaintext: dandet
Key [[11,11],[0,15]] -> Plaintext: zentqn
Key [[11,12],[13,3]] -> Plaintext: therda
Key [[11,15],[21,12]] -> Plaintext: khzthe
Key [[11,17],[8,7]] -> Plaintext: forzgv
Key [[11,25],[14,3]] -> Plaintext: pcxfor
Key [[12,11],[1,0]] -> Plaintext: izzand
Key [[12,15],[3,1]] -> Plaintext: fthemd
Key [[12,15],[3,14]] -> Plaintext: sthezd
Key [[12,19],[3,10]] -> Plaintext: cvforr
```

```
Key [[18,9],[15,0]] -> Plaintext: entenv
Key [[18,11],[11,0]] -> Plaintext: wthend
Key [[18,11],[11,13]] -> Plaintext: jthead
Key [[18,11],[15,0]] -> Plaintext: entind
Key [[18,11],[21,0]] -> Plaintext: ozvand
Key [[18,15],[13,12]] -> Plaintext: cfforn
Key [[18,15],[15,0]] -> Plaintext: entsnx
Key [[18,17],[15,0]] -> Plaintext: entwnf
Key [[18,19],[15,0]] -> Plaintext: entgnz
Key [[18,21],[15,0]] -> Plaintext: entynj
Key [[18,23],[15,0]] -> Plaintext: entonp
Key [[18,25],[15,0]] -> Plaintext: entqnt
Key [[18,25],[25,0]] -> Plaintext: sfbent
Key [[19,1],[15,16]] -> Plaintext: ahxthe
Key [[19,2],[17,15]] -> Plaintext: tforxg
Key [[19,3],[22,15]] -> Plaintext: forbgz
Key [[19,9],[18,17]] -> Plaintext: zandqt
Key [[19,11],[0,15]] -> Plaintext: jentmn
Key [[19,14],[15,3]] -> Plaintext: nhkthe
Key [[19,20],[15,5]] -> Plaintext: thetde
Key [[19,21],[22,7]] -> Plaintext: huthet
Key [[19,23],[10,17]] -> Plaintext: nchfor
Key [[20,1],[5,4]] -> Plaintext: cthevd
Key [[20,1],[5,17]] -> Plaintext: ptheid
Key [[20,1],[13,21]] -> Plaintext: zbzion
Key [[20,3],[13,21]] -> Plaintext: vbtion
Key [[20,5],[13,21]] -> Plaintext: rbnion
Key [[20,5],[25,4]] -> Plaintext: cjforb
Key [[20,7],[13,21]] -> Plaintext: nbhion
Key [[20,9],[13,21]] -> Plaintext: jbbion
Key [[20,11],[13,21]] -> Plaintext: fbvion
Key [[20,11],[19,0]] -> Plaintext: kzpand
Key [[20,13],[13,21]] -> Plaintext: bbpion
Key [[20,15],[13,21]] -> Plaintext: xbjion
Key [[20,17],[13,21]] -> Plaintext: tbdion
Key [[20,19],[13,21]] -> Plaintext: pbxion
Key [[20,21],[13,21]] -> Plaintext: lbrion
Key [[20,23],[13,21]] -> Plaintext: hblion
Key [[20,25],[9,0]] -> Plaintext: yfxent
Key [[20,25],[13,21]] -> Plaintext: dbfion
Key [[21,3],[22,1]] -> Plaintext: fcvfor
Key [[21,4],[7,17]] -> Plaintext: zhcthe
Key [[21,8],[1,19]] -> Plaintext: rfordg
Key [[21,9],[24,17]] -> Plaintext: jandmt
Key [[21,11],[0,15]] -> Plaintext: xentwn
Key [[21,17],[7,4]] -> Plaintext: mhpthe
Key [[21,19],[6,17]] -> Plaintext: forxgr
Key [[21,22],[9,25]] -> Plaintext: theldo
Key [[21,23],[14,25]] -> Plaintext: hmther
Key [[22,11],[17,0]] -> Plaintext: czdand
Key [[22,17],[25,8]] -> Plaintext: otheld
Key [[22,17],[25,21]] -> Plaintext: btheyd
Key [[22,21],[11,22]] -> Plaintext: cxforl
Key [[22,25],[19,0]] -> Plaintext: kfpent
```

```
Key [[22,25],[19,0]] -> Plaintext: kfpent
Key [[23,2],[3,23]] -> Plaintext: flentg
Key [[23,2],[16,23]] -> Plaintext: fyentt
Key [[23,4],[3,7]] -> Plaintext: fzentq
Key [[23,4],[16,7]] -> Plaintext: fmentd
Key [[23,6],[3,17]] -> Plaintext: fventc
Key [[23,6],[16,17]] -> Plaintext: fientp
Key [[23,7],[25,18]] -> Plaintext: ohfthe
Key [[23,8],[3,1]] -> Plaintext: ftenti
Key [[23,8],[16,1]] -> Plaintext: fgentv
Key [[23,9],[4,17]] -> Plaintext: pandut
Key [[23,9],[8,11]] -> Plaintext: vctfor
Key [[23,9],[16,19]] -> Plaintext: forngx
Key [[23,10],[3,11]] -> Plaintext: fxentw
Key [[23,10],[16,11]] -> Plaintext: fkentj
Key [[23,11],[0,15]] -> Plaintext: ventcn
Key [[23,12],[3,21]] -> Plaintext: frento
Key [[23,12],[16,21]] -> Plaintext: feentb
Key [[23,14],[3,5]] -> Plaintext: fjentm
Key [[23,14],[11,23]] -> Plaintext: vforrg
Key [[23,14],[16,5]] -> Plaintext: fwentz
Key [[23,16],[3,15]] -> Plaintext: fdente
Key [[23,16],[16,15]] -> Plaintext: fqentr
Key [[23,18],[3,25]] -> Plaintext: fhents
Key [[23,18],[16,25]] -> Plaintext: fuentf
Key [[23,20],[3,9]] -> Plaintext: ffenty
Key [[23,20],[16,9]] -> Plaintext: fsentl
Key [[23,20],[25,5]] -> Plaintext: bhsthe
Key [[23,22],[3,19]] -> Plaintext: fbentk
Key [[23,22],[16,19]] -> Plaintext: foentx
Key [[23,24],[3,3]] -> Plaintext: fpentu
Key [[23,24],[3,19]] -> Plaintext: thends
Key [[23,24],[16,3]] -> Plaintext: fcenth
Key [[23,24],[25,15]] -> Plaintext: thatve
Key [[23,25],[2,13]] -> Plaintext: ruthat
Key [[23,25],[6,17]] -> Plaintext: hytheh
Key [[24,7],[19,12]] -> Plaintext: ythehd
Key [[24,7],[19,25]] -> Plaintext: ltheud
Key [[24,11],[15,0]] -> Plaintext: eztand
Key [[24,11],[23,14]] -> Plaintext: cbforz
Key [[24,25],[3,0]] -> Plaintext: ufrent
Key [[25,1],[24,9]] -> Plaintext: hothel
Key [[25,9],[10,17]] -> Plaintext: tandit
Key [[25,10],[17,19]] -> Plaintext: lhuthe
Key [[25,11],[0,15]] -> Plaintext: lentgn
Key [[25,15],[20,21]] -> Plaintext: xcjfor
Key [[25,20],[21,1]] -> Plaintext: pforjg
Key [[25,23],[17,6]] -> Plaintext: yhhthe
Key [[25,25],[0,21]] -> Plaintext: forvgn

Brute-force complete. Found 246 potential matches.

┌──(K)(student㉿PC-36)-[~/Documents/K Dhyana/ACL/LAB-2]
└─$
```

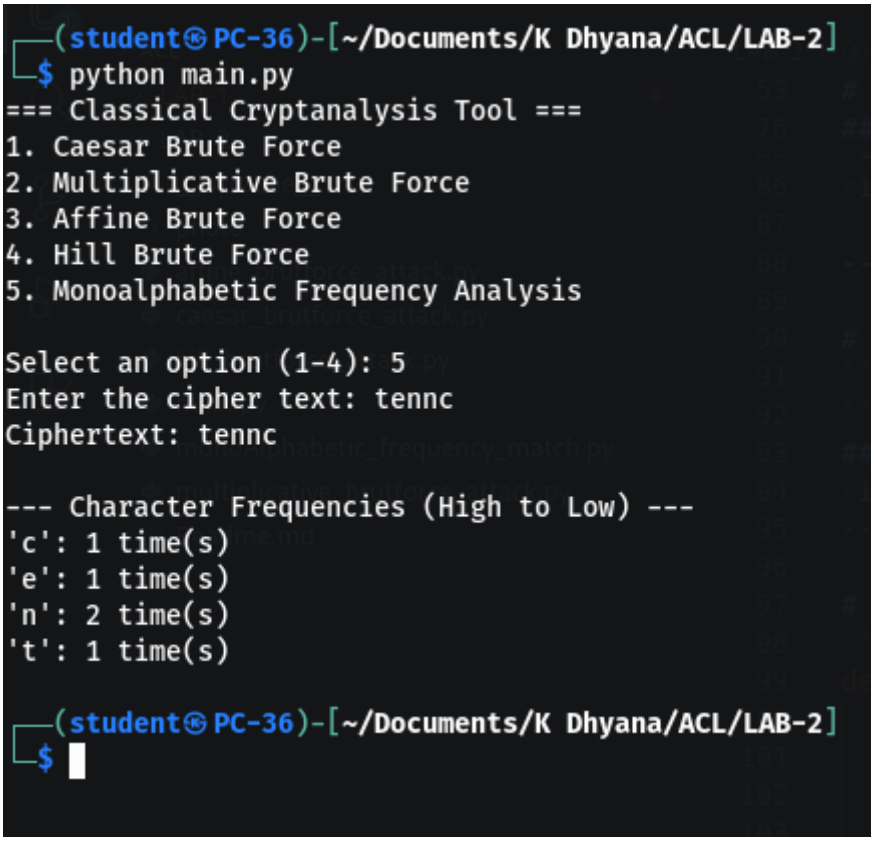
5.Mono-alphabetic Frequency Analysis

```
def frequency(cipher_text:str):
freq = {}
cipher_text = cipher_text.lower()
for char in cipher_text:
if char.isalpha():
freq[char] = freq.get(char, 0) + 1

return freq

def analyze():
cipher_text = input("Enter the cipher text: ")
char_count = frequency(cipher_text)
char_count = sorted(char_count.items())
print(f"Ciphertext: {cipher_text}\n")
print("--- Character Frequencies (High to Low) ---")
for char, count in char_count:
print(f"'{char}': {count} time(s)")
```

Output



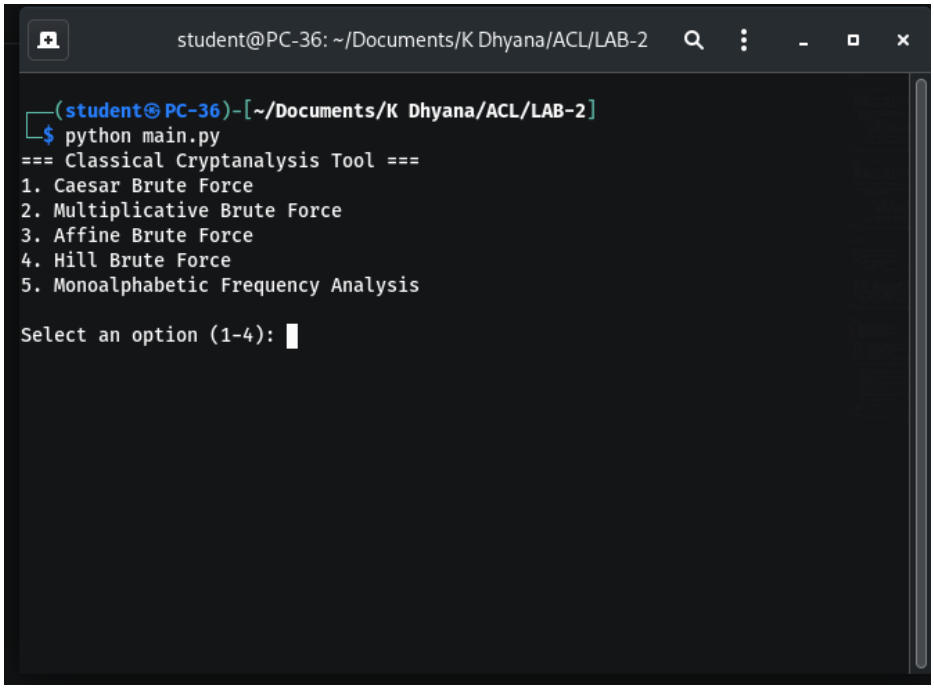
Main Function

```
import caesar_brutforce_attack
import multiplicative_brutforce_attack
import affine_brutforce_attack
import monoAlphabetic_frequency_match
import hill_brutforce_attack

def main():
print("=== Classical Cryptanalysis Tool ===")
print("1. Caesar Brute Force")
print("2. Multiplicative Brute Force")
print("3. Affine Brute Force")
print("4. Hill Brute Force")
print("5. Monoalphabetic Frequency Analysis")
choice = input("\nSelect an option (1-4): ")
if choice == '1':
# Assuming caesar file has a function like bruteforce()
caesar_brutforce_attack.brutforce()
elif choice == '2':
multiplicative_brutforce_attack.brutforce()
elif choice == '3':
affine_brutforce_attack.brutforce()
elif choice == '4':
hill_brutforce_attack.brutforce()
elif choice =='5':
monoAlphabetic_frequency_match.analyze()
else:
print("Invalid selection.")

if __name__ == "__main__":
```

main()


student@PC-36: ~/Documents/K Dhyana/ACL/LAB-2
(student㉿PC-36)-[~/Documents/K Dhyana/ACL/LAB-2]
$ python main.py
=== Classical Cryptanalysis Tool ===
1. Caesar Brute Force
2. Multiplicative Brute Force
3. Affine Brute Force
4. Hill Brute Force
5. Monoalphabetic Frequency Analysis
Select an option (1-4):